

Scanned Code Report

AUDITAGENT

Code Info

Auditor Scan

#	Scan ID 3	✦	Date February 10, 2026
📦	Organization chipi-pay	📦	Repository sessions-smart-contract
📄	Branch main	📄	Commit Hash 6424aa0b...af802805

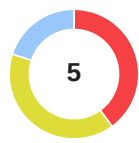
Contracts in scope

src/account.cairo src/lib.cairo tests/test_audit_fixes.cairo tests/test_session_validation.cairo
tests/test_snip9_compatibility.cairo

Code Statistics

⚙️	Findings 5	📄	Contracts Scanned 5	☰	Lines of Code 2125
----	---------------	---	------------------------	---	-----------------------

Findings Summary



Total Findings

- High Risk (2)
- Medium Risk (0)
- Low Risk (2)
- Info (1)
- Best Practices (0)

Code Summary

This protocol implements an advanced, upgradeable StarkNet smart contract account with a primary focus on session key management. It extends the standard OpenZeppelin account contract, enabling users to generate temporary, permissioned keys for interacting with dApps on their behalf. This enhances security and user experience by allowing for gasless transactions or automated actions without exposing the owner's primary private key.

The core feature is the session key system. An account owner can create session keys with a granular set of limitations:

- **Expiration:** Each session key is assigned a `valid_until` timestamp, after which it automatically becomes invalid.
- **Usage Limit:** A `max_calls` counter restricts the total number of transactions a session key can sign before it is exhausted.
- **Function Whitelist:** The owner can define an explicit list of function selectors (`allowed_entrypoints`) that a session key is permitted to call. If this whitelist is left empty, the session key is authorized to call any function except for a hardcoded set of administrative functions.

For security, session keys are explicitly blocked from executing critical, owner-level actions such as upgrading the account contract or managing other session keys, thereby preventing any risk of privilege escalation.

The contract supports two distinct validation paths based on the signature format:

1. **Owner Validation:** Standard transactions signed by the owner's private key are validated using a 2-element signature (`[r, s]`).
2. **Session Key Validation:** Transactions signed by a session key are validated using a 4-element signature (`[session_pubkey, r, s, valid_until]`). The contract first verifies the session's validity (i.e., not expired, calls remaining) and permissions (i.e., the function selector is whitelisted) before proceeding to check the ECDSA signature.

The account is also compatible with SNIP-9 for outside execution (e.g., via paymasters), featuring a custom implementation that enforces session key whitelists before signature validation to secure these interactions.

Entry Points and Actors

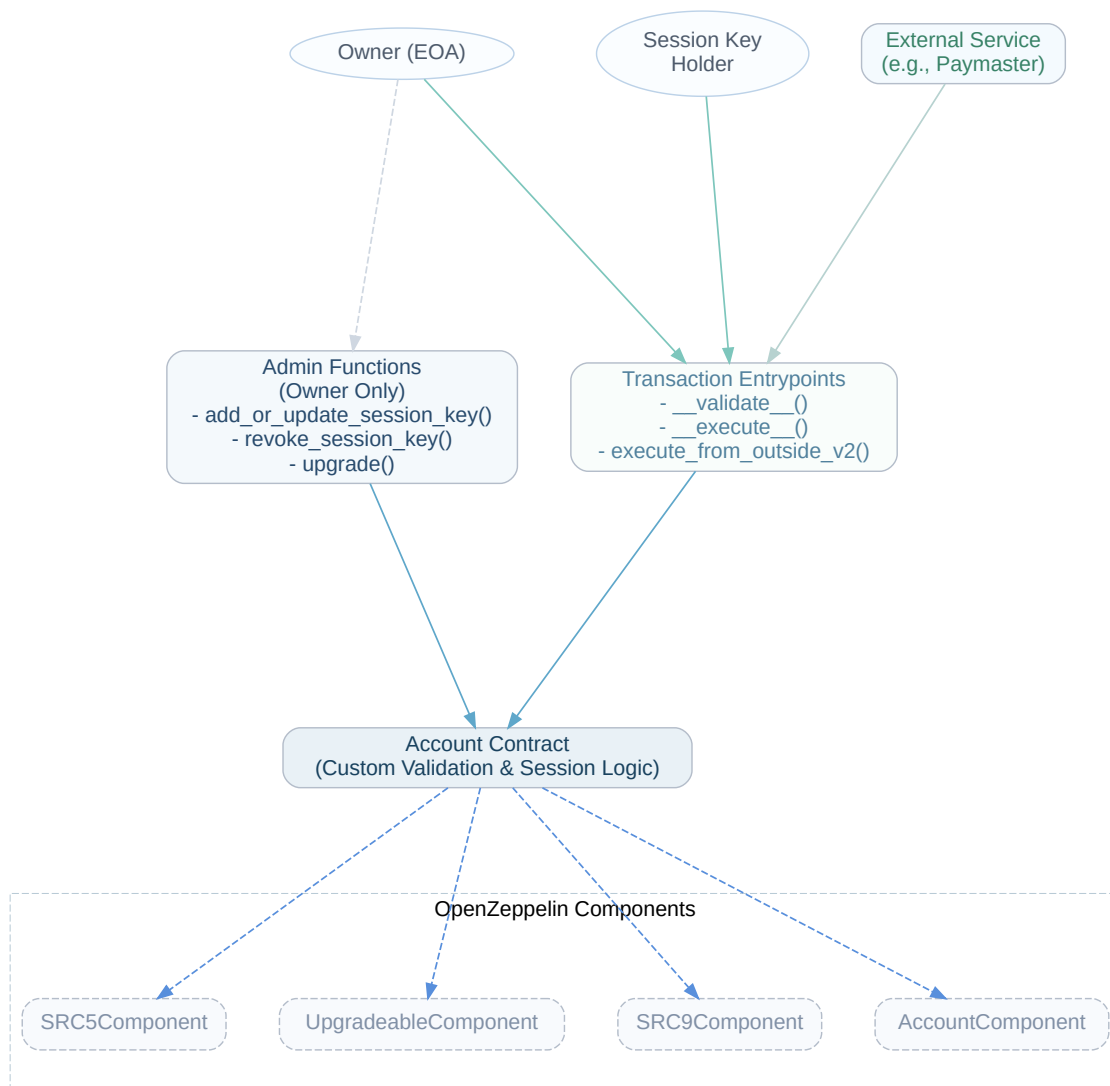
The main actors interacting with the protocol are:

- **Account Owner:** The primary user who controls the account's main public key and manages session keys.
- **Session Key Holder:** An entity, such as a dApp or automated script, that holds a temporary key with delegated permissions.
- **Third-Party Relayer:** An external entity, like a paymaster, that submits pre-signed transactions on behalf of the account via the SNIP-9 outside execution mechanism.

The primary state-modifying entry points for these actors are:

- `__validate__`: The main validation entry point for all transactions, called by the StarkNet OS. The transaction is initiated by either the **Account Owner** or a **Session Key Holder** who provides the signature.
- `__execute__`: The main execution entry point, called by the StarkNet OS after a successful validation. The transaction is initiated by either the **Account Owner** or a **Session Key Holder**.
- `execute_from_outside_v2`: Allows a **Third-Party Relayer** to submit a pre-signed transaction on behalf of the **Account Owner** or a **Session Key Holder**, enabling meta-transactions.

Code Diagram



✦ 1 of 5 Findings

src/account.cairo

Session keys can take over the account by calling OpenZeppelin AccountComponent's `set_public_key` / `setPublicKey` (not included in the admin blocklist)

• High Risk

The contract embeds OpenZeppelin's `AccountComponent::PublicKeyImpl` and `PublicKeyCamelImpl`, which expose privileged entrypoints that rotate the account's owner public key:

```
#[abi(embed_v0)]
impl PublicKeyImpl = AccountComponent::PublicKeyImpl<ContractState>;
#[abi(embed_v0)]
impl PublicKeyCamelImpl = AccountComponent::PublicKeyCamelImpl<ContractState>;
```

Per OpenZeppelin's Account API, `PublicKeyImpl` includes `set_public_key(self, new_public_key, signature)` and `PublicKeyCamelImpl` includes `setPublicKey(self, newPublicKey, signature)`.

This protocol's session restriction logic attempts to prevent session keys from performing owner-level actions by blocking a small set of selectors in `_is_session_allowed_for_calls`:

```
let UPGRADE_SELECTOR: felt252 = selector!("upgrade");
let ADD_SESSION_SELECTOR: felt252 = selector!("add_or_update_session_key");
let REVOKE_SELECTOR: felt252 = selector!("revoke_session_key");
let EXECUTE_SELECTOR: felt252 = selector!("__execute__");
...
if sel == UPGRADE_SELECTOR
|| sel == ADD_SESSION_SELECTOR
|| sel == REVOKE_SELECTOR
|| sel == EXECUTE_SELECTOR {
    return false;
}

// If no whitelist (allowed_entrypoints_len == 0), allow all non-admin selectors
if session.allowed_entrypoints_len == 0 { return true; }
```

Because `set_public_key` / `setPublicKey` are not included in this blocklist, any session that is configured with an "open whitelist" (`allowed_entrypoints_len == 0`, which this contract explicitly treats as allow-all for non-blocklisted selectors) can authorize a transaction that calls `set_public_key` / `setPublicKey`.

This results in a full account takeover:

- A session key holder submits an Invoke tx (or `execute_from_outside_v2`) whose `calls` contains a call to the account itself with selector `set_public_key` (or `setPublicKey`).
- The session passes `_is_session_allowed_for_calls(...)` because the selector is not blocklisted and the whitelist is open (or includes that selector).
- During execution, the account calls itself via `_execute_calls` / `call_contract_syscall`, so inside `set_public_key` the caller is the account itself, satisfying OpenZeppelin's "only self-call via `execute`" requirement.
- The session key holder provides the required `signature` parameter expected by `set_public_key` (the acceptance signature from the new owner key), and can set `new_public_key` to a key they control.

Once the public key is changed, the attacker can produce valid owner signatures and gain full owner-equivalent control of the account, including upgrading the contract and modifying/revoking sessions (via owner path).

Severity Note:

- OZ AccountComponent::set_public_key/setPublicKey are only-self callable (standard OZ behavior).
- Self-calls via **execute** result in caller == account, satisfying only-self requirement.
- Some users will configure sessions with allowed_entrypoints_len == 0 as intended by design.
- The acceptance signature required by set_public_key can be produced by the attacker controlling the new key.

✦ 2 of 5 Findings

tests/test_audit_fixes.cairo

Session keys can take over the account by calling OpenZeppelin set_public_key / setPublicKey (missing from admin selector blacklist)

• High Risk

The session enforcement blacklist in `_is_session_allowed_for_calls` only blocks `{upgrade, add_or_update_session_key, revoke_session_key, __execute__}`. However, the contract embeds OpenZeppelin `AccountComponent::PublicKeyImpl` and `PublicKeyCamelImpl`, which expose owner-rotation entrypoints (`set_public_key` and `setPublicKey`) that are intended to be callable only via self-call through `__execute__`.

Because session transactions ultimately execute calls via the account contract itself, a session key with an **empty whitelist** (`allowed_entrypoints_len == 0`, meaning "allow all non-admin selectors") can include a call targeting the account contract's public key rotation entrypoint. Since that selector is not blocked, `__validate__` accepts the session transaction, and `__execute__` performs the self-call; the OZ public key rotation function's `assert_only_self` requirement is satisfied (caller is the account), and the attacker can provide the required "new owner acceptance" signature using their own key.

Vulnerable logic (incomplete admin blacklist + allow-all behavior):

```
fn _is_session_allowed_for_calls(
    self: @ContractState,
    session_key: felt252,
    calls: Span<Call>
) -> bool {
    // ...
    let UPGRADE_SELECTOR: felt252 = selector!("upgrade");
    let ADD_SESSION_SELECTOR: felt252 = selector!("add_or_update_session_key");
    let REVOKE_SESSION_SELECTOR: felt252 = selector!("revoke_session_key");
    let EXECUTE_SELECTOR: felt252 = selector!("__execute__");

    // First pass: check for blocked admin selectors
    // (NOTE: does NOT include set_public_key / setPublicKey)
    // ...

    // If no whitelist, allow all non-admin selectors
    if session.allowed_entrypoints_len == 0 { return true; }

    // ... whitelist checks
}
```

Concrete exploit scenario:

- 1) Account owner creates a session with `allowed_entrypoints = []` (allow-all non-admin), expecting it cannot escalate privileges (as tested in `test_audit3_session_blocked_from_add_session`).
- 2) Attacker obtains/controls the session key (common in dapp-session models).
- 3) Attacker submits a session-signed transaction whose `calls` includes:
 - `to = account`, `selector = selector!("set_public_key")` (or `selector!("setPublicKey")`),
`calldata = [attacker_new_pubkey, attacker_acceptance_signature...]`.
- 4) `__validate__` accepts because the selector is not on the blacklist and empty whitelist returns `true`.
- 5) `__execute__` self-calls into `set_public_key`, which passes OZ `assert_only_self` and updates the account's

owner public key to the attacker's key.

6) With owner key control, the attacker can subsequently sign normal 2-element owner transactions to drain tokens, add unrestricted sessions, upgrade the account, etc.

Impact:

- **Full account takeover** from a session key that was intended to be restricted from admin capabilities.
- Leads to **theft/loss of all assets controlled by the account** (ERC20/721/1155 holdings, approvals, future deposits), and persistent compromise (attacker becomes the owner key).

This is especially dangerous because it affects the common "empty whitelist = allow all user actions" configuration, making such sessions equivalent to full ownership.

Severity Note:

- The account integrates OZ AccountComponent::PublicKeyImpl/PublicKeyCamellImpl with set_public_key/setPublicKey guarded by assert_only_self.
- **execute** performs self-calls so that assert_only_self is satisfied when calling the account.
- Real deployments commonly create sessions with empty whitelist semantics (allow all non-admin selectors).
- Any required acceptance signature for owner rotation can be produced by the attacker for their own new key.

✦ 3 of 5 Findings

src/account.cairo

Double-consumption of session calls within a single transaction via nested execute_from_outside_v2

• Low Risk

The contract increments the session call counter in two separate places: once in the standard transaction path (**validate**) and once in the SNIP-9 path (`execute_from_outside_v2`). Because `execute_from_outside_v2` is not in the admin blocklist enforced by `_is_session_allowed_for_calls`, a session-authenticated standard transaction can include a call to the account's own `execute_from_outside_v2`. This causes the same session to be charged twice in a single chain transaction, violating the invariant that a successful transaction authenticated with a session key increments `calls_used` by exactly one regardless of the number of calls in the multicall.

Relevant code:

```
// __validate__ (standard tx path) – consumes 1 on success
if signature.len() == 4 {
    ...
    if check_ecdsa_signature(msg_hash, session_pubkey, r, s) {
        self._consume_session_call(session_pubkey); // <-- first increment
        return starknet::VALIDATED;
    }
}
```

```
// execute_from_outside_v2 (SNIP-9 path) – also consumes 1 on success
if signature.len() == 4 {
    is_session_sig = true;
    session_pubkey = *signature.at(0);
    assert(self._is_session_allowed_for_calls(session_pubkey, outside_execution.calls),
'Session: unauthorized selector');
}
...
assert(is_valid_signature, 'SRC9: invalid signature');
if is_session_sig {
    self._consume_session_call(session_pubkey); // <-- second increment (same tx)
}
```

```
// Session allow-check – admin blocklist does NOT include execute_from_outside_v2
let UPGRADE_SELECTOR: felt252 = selector!("upgrade");
let ADD_SESSION_SELECTOR: felt252 = selector!("add_or_update_session_key");
let REVOKE_SESSION_SELECTOR: felt252 = selector!("revoke_session_key");
let EXECUTE_SELECTOR: felt252 = selector!("__execute__");
...
if sel == UPGRADE_SELECTOR
|| sel == ADD_SESSION_SELECTOR
|| sel == REVOKE_SESSION_SELECTOR
|| sel == EXECUTE_SELECTOR { return false; }
// execute_from_outside_v2 is not blocked here
```

Proof-of-concept scenario:

- Create a session with `allowed_entrypoints_len == 0` (allow all non-admin) or explicitly whitelist selector! ("execute_from_outside_v2").
- Submit a standard session-signed transaction (validated via **validate**) whose calls array includes a Call to this account's `execute_from_outside_v2` with a valid 4-element session signature for its `OutsideExecution` payload.
- Outcome: **validate** increments `calls_used` once; the nested `execute_from_outside_v2` call increments it again; `calls_used_after == calls_used_before + 2`, breaking the invariant.

✦ 4 of 5 Findings

src/account.cairo

Missing SRC-5 Interface Registration Breaks Paymaster Discovery

• Low Risk

Root cause: The Account contract embeds `SRC5Component/SRC5Impl` (exposing `supports_interface()`), but the constructor only initializes the `AccountComponent` and `SRC9Component` and does not register any SRC-5 interface IDs. As a result, `supports_interface()` will report false for all interface ID queries even though the account implements the relevant entrypoints.

Exploit path: (1) A paymaster or integrator performs standards-based discovery via `supports_interface()` to detect support for the session key manager interface and the SNIP-9/SRC9 v2 outside-execution interface, (2) these checks return false because no interface IDs were registered, (3) the integrator treats the account as unsupported, falling back to non-standard ABI inspection or failing integration.

Impact: Interoperability/standards compliance failure that breaks documented paymaster discovery guarantees and SNIP requirements for SRC-5 registration; does not directly compromise on-chain authorization but can cause ecosystem integration failure.

Recommended fix: In the constructor, register the relevant interface IDs via `self.src5.register_interface(SESSION_KEY_MANAGER_ID)` and `self.src5.register_interface(ISRC9_V2_ID)`. Compute these IDs using the standard Cairo/OZ convention (XOR of function selectors):
`SESSION_KEY_MANAGER_ID = selector!("add_or_update_session_key") ^ selector!("revoke_session_key") ^ selector!("get_session_data")`; `ISRC9_V2_ID = selector!("execute_from_outside_v2") ^ selector!("is_valid_outside_execution_nonce")`. Add regression tests that call `supports_interface()` and assert it returns true for these two IDs and false for an unregistered random ID.

Severity Note:

- Some paymasters/integrators use SRC-5 `supports_interface()` to gate compatibility for session key manager and `ISRC9_V2`.
- They will not implement their own ABI probing fallback or will fail closed when SRC-5 returns false.

✦ 5 of 5 Findings

src/account.cairo

Malleable `valid_until` in session signatures for `execute_from_outside_v2`[• Info](#)

The `execute_from_outside_v2` function supports meta-transactions signed by a session key using a 4-element signature: `[session_pubkey, r, s, valid_until]`. The function computes a message hash based on the `OutsideExecution` struct and validates the signature `(r, s)` against it. However, the `valid_until` timestamp, which is passed as the fourth element of the signature array, is not included in the signed message hash.

The `is_valid_signature` function, called by `execute_from_outside_v2`, checks this `valid_until` value against the block timestamp to enforce an expiration on the transaction. Because this value is not part of the signed data, a malicious relayer can modify it without invalidating the core signature components `(r, s)`.

This allows a relayer to extend the validity of a signed `OutsideExecution` message, bypassing the user's intended expiration. An attacker can intercept a valid signed transaction, replace the `valid_until` timestamp with a much later one, and execute the transaction at a time that is unfavorable to the user, such as after significant market price changes for a swap transaction.

This vulnerability undermines a critical security feature of the session key system, as users can no longer rely on per-transaction expiries for meta-transactions.

Code Snippet from `execute_from_outside_v2`:

```
// The message hash is computed from the `outside_execution` struct, which does not include
the `valid_until` from the signature.
let oz_hash = outside_execution.get_message_hash(get_contract_address());
let is_valid_oz = SRC6Impl::is_valid_signature(@self, oz_hash, sig_copy1);
```

Code Snippet from `is_valid_signature`:

```
if signature.len() == 4 {  
    // ...  
    let valid_until: u64 = match (*signature.at(3)).try_into() {  
        Option::Some(v) => v,  
        Option::None => { return 0; } // Invalid timestamp format  
    };  
  
    // This check uses the `valid_until` from the signature, which is malleable.  
    if get_block_timestamp() > valid_until {  
        return 0;  
    }  
  
    // ...  
  
    // The signature verification uses a hash that is independent of `valid_until`.  
    let is_valid = check_ecdsa_signature(  
        hash,  
        session_pubkey,  
        r,  
        s  
    );  
    // ...  
}
```

This issue is distinct from the `__validate__` flow, where the `_session_message_hash` function correctly includes the `valid_until` timestamp in the signed hash, preventing such malleability.

Disclaimer

Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.